

Представление чисел в ЭВМ

Двоичная арифметика и машинная точность

1 Целые числа

Двоичное кодирование

В ЭВМ вся информация хранится в виде последовательности **битов**. Каждый бит может принимать только два значения:

$$0 \quad \text{или} \quad 1.$$

Следовательно, с помощью p бит можно закодировать

$$2^p$$

различных последовательностей.

Если интерпретировать последовательность

$$a_{p-1}a_{p-2} \dots a_1a_0, \quad a_i \in \{0, 1\},$$

как двоичное число, то её значение равно

$$A = a_{p-1}2^{p-1} + a_{p-2}2^{p-2} + \dots + a_12 + a_0 = \sum_{i=0}^{p-1} a_i2^i.$$

Таким способом кодируются беззнаковые целые числа

$$0, 1, \dots, 2^p - 1.$$

Например, для $p = 32$ имеется

$$2^{32}$$

различных битовых последовательностей.

Для стандартного 32-битного целого числа

$$4 \text{ байта} \times 8 \text{ бит} = 32 \text{ бита}.$$

Отрицательные числа

Для представления целых чисел со знаком необходимо закодировать как положительные, так и отрицательные значения.

В дополнительном коде отрицательное число $-N$ представляется числом

$$2^p - N.$$

Иными словами, код числа $-N$ выбирается так, чтобы

$$N + (2^p - N) = 2^p \equiv 0 \pmod{2^p}.$$

Поэтому арифметику p -битных целых чисел удобно рассматривать как арифметику по модулю 2^p .

Например,

$$-1 \longleftrightarrow 2^p - 1 = \underbrace{11 \dots 11}_p \text{ бит}.$$

Диапазон знаковых p -битных целых чисел имеет вид

$$-2^{p-1} \leq N \leq 2^{p-1} - 1.$$

Замечание. Именно поэтому при переполнении целочисленной переменной результат фактически “переходит через границу” доступного диапазона.

2 Числа с плавающей точкой

Фиксированная и плавающая точка

Вещественное число можно хранить в форме с **фиксированной точкой**, заранее задавая положение десятичного разделителя.

Например,

$$1,031 = \frac{1031}{1000}, \quad 10,05 = \frac{1005}{100}.$$

Такой способ неудобен для чисел, изменяющихся в очень широком диапазоне. Поэтому на практике используется **форма с плавающей точкой**:

$$x = M \cdot b^E,$$

где M — мантисса, E — порядок, а b — основание системы счисления.

Например, в десятичной системе

$$1,031 = 1,031 \cdot 10^0,$$

$$10,51 = 1,051 \cdot 10^1.$$

Нормализованная форма

Чтобы запись числа с плавающей точкой была однозначной, используется **нормализованная форма**: положение запятой выбирается так, чтобы перед ней находилась ровно одна ненулевая цифра.

Например,

$$1,031 \cdot 10^0$$

является нормализованной записью, тогда как

$$10,31 \cdot 10^{-1}$$

описывает то же число, но нормализованной записью не является.

В двоичной системе первая цифра нормализованной мантиисы всегда равна единице. Поэтому число можно записать в форме

$$x = (-1)^S \left(1 + \frac{M}{2^n} \right) 2^E,$$

где

$$S \in \{0, 1\}, \quad M = 0, 1, \dots, 2^n - 1.$$

Здесь

- S — бит знака;
- M — целое число, кодирующее дробную часть мантиисы;
- n — число бит дробной части мантиисы;
- E — порядок.

Поскольку старшая единица нормализованной мантиисы всегда известна, её нет необходимости явно хранить в памяти. Это так называемая **скрытая единица**.

Формат IEEE 754

В памяти нормализованное число записывается в виде трёх полей:

$$\boxed{S} \quad \boxed{E_S} \quad \boxed{M},$$

где S — знак, E_S — смещённый порядок, M — дробная часть мантиисы.

Если под порядок выделено w бит, вводится смещение

$$B = 2^{w-1} - 1.$$

Тогда вместо самого порядка E хранится

$$E_S = E + B = E + 2^{w-1} - 1.$$

Следовательно,

$$E = E_S - (2^{w-1} - 1),$$

и нормализованное число принимает вид

$$x = (-1)^S \left(1 + \frac{M}{2^n}\right) 2^{E_S - (2^{w-1} - 1)}.$$

Два значения поля порядка зарезервированы для специальных случаев:

$$E_S = 0, \quad E_S = 2^w - 1.$$

Поэтому для нормализованных чисел

$$1 \leq E_S \leq 2^w - 2.$$

Подставляя

$$E_S = E + 2^{w-1} - 1,$$

получаем

$$\begin{aligned} 1 &\leq E + 2^{w-1} - 1 \leq 2^w - 2, \\ -2^{w-1} + 2 &\leq E \leq 2^{w-1} - 1. \end{aligned}$$

Таким образом,

$$E_{\min} = -2^{w-1} + 2, \quad E_{\max} = 2^{w-1} - 1.$$

Одинарная точность: float

Для формата IEEE 754 binary32, которому обычно соответствует тип `float`,

$$32 = 1 + 8 + 23.$$

Биты распределяются следующим образом:

$$\underbrace{\boxed{S}}_{1 \text{ бит}} \quad \underbrace{\boxed{E_S}}_{8 \text{ бит}} \quad \underbrace{\boxed{M}}_{23 \text{ бита}}.$$

То есть

$$w = 8, \quad n = 23, \quad B = 2^7 - 1 = 127.$$

Отсюда

$$E_{\min} = -126, \quad E_{\max} = 127.$$

Минимальное положительное нормализованное число получается при $M = 0$ и $E = E_{\min}$:

$$f_0 = 2^{E_{\min}} = 2^{-126} \approx 1.18 \cdot 10^{-38}.$$

Следующее машинное число имеет $M = 1$:

$$f_1 = \left(1 + \frac{1}{2^n}\right) 2^{E_{\min}}.$$

Расстояние между ними равно

$$f_1 - f_0 = 2^{E_{\min} - n}.$$

При этом

$$\frac{f_0}{f_1 - f_0} = 2^n.$$

Для float, где $n = 23$,

$$2^{23} \approx 8.4 \cdot 10^6.$$

То есть без дополнительного механизма около нуля возникла бы очень большая область непредставимых чисел.

Денормализованные числа

Чтобы заполнить область около нуля, вводятся **денормализованные** (subnormal) числа.

Для них поле порядка полностью заполнено нулями:

$$E_S = 0.$$

Скрытая единица в мантиссе при этом отсутствует, поэтому число записывается как

$$x = (-1)^S \frac{M}{2^n} 2^{E_{\min}}.$$

Минимальное положительное денормализованное число соответствует $M = 1$:

$$f_{\min}^{\text{denorm}} = 2^{E_{\min} - n}.$$

Максимальное денормализованное число:

$$f_{\max}^{\text{denorm}} = \left(1 - \frac{1}{2^n}\right) 2^{E_{\min}}.$$

А следующее за ним число уже является минимальным нормализованным:

$$f_{\min}^{\text{norm}} = 2^{E_{\min}}.$$

Таким образом, переход от денормализованных чисел к нормализованным происходит без большого разрыва.

Специальные значения

Зарезервированные значения поля порядка позволяют кодировать ноль, бесконечность и NaN.

Для краткости обозначим через $00 \dots 0$ последовательность из нулей, а через $11 \dots 1$ — последовательность из единиц.

1.

$$\boxed{S} \boxed{00 \dots 0} \boxed{00 \dots 0}$$

соответствует числу $+0$ или -0 в зависимости от S .

2.

$$\boxed{S} \boxed{00 \dots 0} \boxed{M}, \quad M \neq 0,$$

соответствует денормализованному числу.

3.

$$\boxed{S} \boxed{11 \dots 1} \boxed{00 \dots 0}$$

соответствует

$$+\infty \quad \text{или} \quad -\infty.$$

4.

$$\boxed{S} \boxed{11 \dots 1} \boxed{M}, \quad M \neq 0,$$

соответствует NaN (*Not a Number*).

5. При

$$1 \leq E_S \leq 2^w - 2$$

кодируется обычное нормализованное число.

Машинная точность

Поскольку число бит мантииссы конечно, результат арифметической операции в общем случае приходится округлять до ближайшего представимого машинного числа.

Например,

$$1,08 \cdot 2,03 = 2,1924.$$

Если оставить только три значащие цифры, получим

$$2,1924 \longrightarrow 2,19,$$

что вносит относительную погрешность порядка

$$10^{-3}.$$

Рассмотрим теперь машинные числа около единицы. При $E = 0$ они имеют вид

$$f = 1 + \frac{M}{2^n}.$$

Два соседних числа:

$$f_0 = 1, \quad f_1 = 1 + \frac{1}{2^n}.$$

Расстояние между ними равно

$$f_1 - f_0 = \frac{1}{2^n}.$$

Эта величина называется **машинным эпсилоном**:

$$\boxed{\varepsilon = 2^{-n}}.$$

Она характеризует относительное расстояние между соседними машинными числами порядка единицы.

При округлении к ближайшему числу максимальная относительная погрешность имеет порядок

$$\frac{\varepsilon}{2} = 2^{-(n+1)}.$$

